

Severity vs. Priority Justification Note

Document ID: SP-OHRM-001 Version: 1.0 Author: QA Lead Date: 2026-07-20

Working definitions used on this engagement: Severity = technical/business impact when the defect occurs (how bad is it?). Assessed by QA, does not change with schedule. Priority = urgency of fixing it relative to everything else (how soon?). Assessed with the product owner; considers frequency, workaround, exposure, and release timing.

The two deliberately diverge on half of these defects — that divergence is the point of rating them separately.

DEF-OHRM-001 — Login 500s

SEVERITY: CRITICAL · PRIORITY: P2

Why Critical severity: when it occurs, there is no degraded mode — the entire application is unreachable for every user. Total loss of function is the definition of Critical, regardless of how often it happens.

Why only P2 priority: the failure is intermittent (~1 in 10–15 loads under sustained activity), self-recovers within seconds, and has a trivial user workaround (retry). Evidence points at shared-tenant infrastructure rather than application code, so the fix path is server-log correlation and capacity/rate-limit tuning, not an emergency code patch. It must be root-caused before any production go-live — but it does not justify stopping current development the way a P1 would.

This is the classic "Critical severity, non-P1 priority" case: catastrophic when it fires, but rare, transient, and workaround-able.

DEF-OHRM-002 — Unhandled logout TypeError

SEVERITY: LOW · PRIORITY: P3

Why Low severity: no user-visible malfunction — logout completes, data is intact; the damage is an uncaught exception in the console. Functional impact is nil.

Why P3 rather than P4: two graduated reasons. First, uncaught exceptions pollute error monitoring: in production this would fire alerts and bury genuinely new errors (it already forced the automation to add a targeted exception suppression, which is test-side debt). Second, an interceptor that dereferences an undefined response is a latent robustness gap — the same code path could swallow a real failed-request error. Cheap to fix (a null guard), worth scheduling, not worth expediting.

DEF-OHRM-003 — Invalid leave date range accepted

SEVERITY: HIGH · PRIORITY: P2

Why High (not Critical) severity: it silently corrupts business data — leave requests with inverted ranges enter the workflow with nonsensical durations, which can distort leave balances and downstream approvals. Data-integrity corruption in a core HR process is High. It stops short of Critical because it does not block any function and affects only records created through one specific input path.

Why P2 priority: the broken path (manual keyboard entry) is a common, realistic behaviour — users type dates — so exposure is real, and cleanup cost grows with every bad record created. It should go into the next fix cycle and is flagged as a go-live blocker in the closure recommendation. It is not P1 because there is a functioning validated path (the calendar picker), approvers act as a human backstop, and no committed release is currently in jeopardy. If a production deployment date were set, this would be re-triaged to P1.

DEF-OHRM-004 — Stale employee name header

SEVERITY: MEDIUM · PRIORITY: P3

Why Medium severity: the saved data is correct (persistence verified — TC-PIM-007 passes on reload); the defect is a stale UI projection. But it is not merely cosmetic: an HR user who just corrected a name and still sees the old one will reasonably conclude the save failed, retry it, or lose trust in the record — in an HR system, perceived data incorrectness has real support-ticket cost. Correct data + misleading display lands squarely in Medium.

Why P3 priority: consistent and easy to demo, but confined to one screen, with an obvious recovery (refresh) and zero data loss. Fix in normal course; bundle with other PIM UI work. Also carries a scheduled QA action: add TC-PIM-013 so the regression suite guards the fix.

Summary matrix

DEFECT	SEVERITY (IMPACT)	PRIORITY (URGENCY)	ONE-LINE RATIONALE FOR THE GAP
DEF-OHRM-001	Critical	P2	Catastrophic when it fires, but rare, transient, retry-able, and likely infra not code
DEF-OHRM-002	Low	P3	No functional impact, but pollutes monitoring and hides a latent error-handling gap
DEF-OHRM-003	High	P2	Silent data corruption in a core flow; validated alternate path keeps it out of P1 — becomes P1 on a release date
DEF-OHRM-004	Medium	P3	Data correct, display wrong; user-trust cost is real but recovery is trivial